100+ Next.js Interview Questions With Answers

Comprehensive Next.js Interview Preparation

Question 1 INTERMEDIATE

What's the difference between searchParams and params in page props?

Answer:

params contains dynamic route segments; searchParams contains URL query parameters

Examples:

- **URL:** /blog/post-1?sort=date
- **params:** { slug: 'post-1' }
- **searchParams:** { sort: 'date' }

Question 2 ADVANCED

What's wrong with this environment variable usage?

```
'use client'

export default function Component() {
  const apiKey = process.env.API_SECRET_KEY;
  return <div>API: {apiKey}</div>;
}
```

Security Issue:

Exposing secret in Client Component bundles it for browser access

Explanation: Client Components expose all code to browser. Secrets must stay server-only.

Fix: Use NEXT_PUBLIC_ prefix for client vars or fetch from Server Component/API

Question 3 ADVANCED

Why doesn't this cookie update work?

```
export default async function Page() {
  const cookieStore = cookies();
  cookieStore.set('theme', 'dark');
  return <div>Theme updated</div>;
}
```

Error:

Cannot set cookies in Server Component page - only in Server Actions or Route Handlers

Explanation: Pages are read-only for request data. Cookie mutations require Server Actions or API routes.

Fix: Move cookie.set() to Server Action and call from form or button

Question 4 ADVANCED

Why does this cause request timeout?

```
export async function GET() {
  const results = [];
  for (let i = 0; i < 100; i++) {
    const data = await fetch(`https://api.example.com/${i}`);
    results.push(await data.json());
  }
  return Response.json(results);
}
```

Issue:

Sequential awaits create slow waterfall; exceeds serverless timeout

Explanation: 100 sequential requests take too long. Should parallelize with Promise.all or batch.

Fix: Use Promise.all with mapped fetch array for parallel requests

Question 5 ADVANCED

What's wrong with this Link prefetch behavior?

```
import Link from 'next/link';

export default function Nav() {
  return (
    <>
      {items.map(item => (
        <Link href={item.href} prefetch={true}>
            {item.label}
        </Link>
      ))}
    </>
  );
}
```

Issue:

Prefetching all links on render wastes bandwidth for unused routes

Explanation: Default prefetch behavior is smarter. Only enable for critical next-step routes.
It also increases your cost for example on vercel. So, avoid prefetching all links.

Fix: Use default prefetch or prefetch={false} for less-used links

Question 6 INTERMEDIATE

What's the purpose of the Image component's priority prop?

Answer:

Disables lazy loading and preloads image for LCP (Largest Contentful Paint)

When to Use:

- Above-the-fold hero images
- LCP elements that are images
- Critical images visible on load
- Use sparingly - only 1-2 per page

It adds preload link tag and fetchpriority="high" attribute for faster loading.

Question 7 ADVANCED

What's wrong with this Server Action?

```
'use server'

export async function updateUser(userId, data) {
  const user =
    await db.user.update({
      where: { id: userId },
      data: data,
    });

  redirect('/dashboard');
  return user;
}
```

Issue:

Code after redirect() never executes; return statement is unreachable

Explanation: redirect() throws an error internally to break execution flow. Any code after it will never run.

Fix: Remove return statement after redirect() or use it as the last statement

Question 8 INTERMEDIATE

What's the difference between next build and next build --profile?

Answer:

--profile flag enables React profiling for analyzing component performance

Usage:

- Generates production build with profiling
- Use React DevTools Profiler in production
- Adds minimal overhead
- Useful for performance debugging
- Don't use for actual production deployment

Question 9 ADVANCED

Why does this component cause a hydration error?

```
'use client'

export default function Component() {
  const time = new Date().toLocaleTimeString();

  return <div>Current time: {time}</div>;
}
```

Error:

Hydration mismatch - server and client render different times

Explanation: Server renders at build/request time, client hydrates later. Time values differ, causing mismatch.

Fix: Use useEffect to set time only on client, or suppress hydration warning with suppressHydrationWarning

Question 10 ADVANCED

What happens when you export this config?

```
export const dynamic = 'force-static';
export const revalidate = 60;

export default async function Page() {
  const data = await fetch(
    'some_url',
    { cache: 'no-store' }
  );
}
```

Result:

Conflicting configurations (static vs dynamic)

Explanation: You will not get any error however, force-static requires all data to be cached, but cache: 'no-store' makes it dynamic.

Fix: Use dynamic = 'force-dynamic' or remove cache: 'no-store'

Question 11 INTERMEDIATE

How do you handle errors in the app directory?

Answer:

Use error.js (Client Component) and global-error.js for root layout errors

Error Boundaries:

- **error.js:** Handles errors in route segment
- **global-error.js:** Catches root layout errors
- **not-found.js:** Handles 404 errors
- Must be Client Components with 'use client'
- Receives error and reset props

Question 12 ADVANCED

What's the issue with this Server Action form?

```
async function createPost(formData) {
  'use server'

  const title = formData.get('title');

  const post =
    await db.post.create({
      data: { title }
    });

  revalidatePath('/posts');
  return post;
}
```

Issue:

Returning non-serializable data (database object) from Server Action

Explanation: Server Actions can only return serializable data. Database objects often contain methods, symbols, or circular refs.

Fix: Return plain object: return { id: post.id, title: post.title }

Question 13 ADVANCED

Why doesn't this middleware redirect work?

```
export function middleware(request) {
  const token = request.cookies.get('token');

  if (!token) {
    redirect('/login');
  }
}
```

Error:

redirect() doesn't work in middleware; must use NextResponse.redirect()

Explanation: Middleware runs at the edge and uses Web APIs, not Next.js functions like redirect().

Fix: return NextResponse.redirect(new URL('/login', request.url))

Question 14 INTERMEDIATE

What's the purpose of generateStaticParams?

Answer:

Generates dynamic route segments at build time for static generation (SSG)

Usage:

- Replaces getStaticPaths from pages directory
- Returns array of param objects
- Enables static generation of dynamic routes
- Can be used with generateMetadata
- Runs at build time for production

Question 15 INTERMEDIATE

How do you implement Incremental Static Regeneration (ISR) in Next.js 13+?

Answer:

Use revalidate option in fetch() or export revalidate from page

Methods:

- **Per-fetch:** fetch(url, { next: { revalidate: 60 } })
- **Per-route:** export const revalidate = 60
- **On-demand:** revalidatePath() or revalidateTag()
- Time in seconds; 0 = no cache

Question 16 ADVANCED

What's the problem with this streaming implementation?

```
export default async function Page() {
  const data = await fetchData();

  return (
    <Suspense fallback={<Loading />}>
      <div>{data.title}</div>
    </Suspense>
  );
}
```

Issue:

Data is awaited before render; Suspense boundary doesn't stream

Explanation: When you await at page level, entire page waits. Suspense needs async child component to work properly.

Fix: Move async data fetch(fetchData line) to separate component inside Suspense

Question 17 ADVANCED

What's wrong with this optimization config?

```
// next.config.js
module.exports = {
  experimental: { appDir: true },
  images: {
    domains: ['*.cloudinary.com']
  }
};
```

Error:

Wildcards not supported in domains config; appDir no longer experimental

Explanation: domains doesn't support wildcards. Also appDir is stable in 13.4+, not experimental.

Fix: Use remotePatterns with hostname pattern matching instead

Question 18 ADVANCED

What's the problem with this font optimization?

```
import { Inter } from 'next/font/google';

export default function Page() {
  const inter = Inter({ subsets: ['latin'] });

  return (
    <div className={inter.className}>
      Hello World
    </div>
  );
}
```

Issue:

Font loaded in component causes new instance on each render

Explanation: Font should be loaded at module level, not inside component. Creates multiple font loads.

Fix: Move font import outside component: const inter = Inter({...}) at top level or inside layout.js file

Question 19 INTERMEDIATE

How do you access route parameters in Server Components vs Client Components?

Answer:

Server Components receive params as props; Client Components need it passed from Server Component

Server Component:

- Direct access via params prop
- async function Page({ params })

Client Component:

- No direct params access
- Must be passed from parent Server Component
- Or use usePathname/useSearchParams for URLs

Question 20 ADVANCED

What's the issue with this streaming setup?

```
export default async function Layout({ children }) {
  const user = await getUser();

  return (
    <div>
      <Header user={user} />
      {children}
    </div>
  );
}
```

Issue:

Layout awaits user data, blocking all child pages from streaming

Explanation: Layouts block all nested content. Async operations in layouts prevent streaming benefits.

Fix: Move user fetch to Header component wrapped in Suspense

Question 21 INTERMEDIATE

How do you handle redirects in Server Actions vs Route Handlers?

> **Answer:**
> Server Actions use redirect(), Route Handlers use NextResponse.redirect()

Server Actions:

- Use redirect('/path') directly
- Throws error to break execution

Route Handlers:

- Return NextResponse.redirect(url)
- Can set status codes (301, 302, etc.)

Question 22 INTERMEDIATE

What's the purpose of the unstable_cache function?

> **Answer:**
> Caches results of expensive operations (like database queries) across requests

Features:

- Works with any async function, not just fetch
- Supports cache tags for revalidation
- Can set custom revalidate times
- Useful for database queries, computations
- Name includes "unstable" but widely used

Question 23 INTERMEDIATE

How do you implement optimistic UI updates with Server Actions?

Answer:

Use useOptimistic hook to show immediate UI updates before server response

Implementation:

- Import useOptimistic from React
- Pass current state and update function
- Update UI immediately
- Rollback on error automatically
- Sync with server response when complete

Question 24 ADVANCED

What's wrong the below code?

```
'use client'

export default function Form() {
  const handleSubmit = async (e) => {
    e.preventDefault();
    await submitAction();
  };
  return <form onSubmit={handleSubmit}>...</form>;
}
```

Issue:

Form won't work without JavaScript; not progressively enhanced

Explanation: Server Actions work with or without JS when used with action prop, but onSubmit requires JS.

Fix: Use action={submitAction} prop instead of onSubmit handler

Question 25 ADVANCED

Why does this cause a waterfall of requests?

```
export default async function Page() {
  const user = await getUser();
  const posts = await getPosts(user.id);
  const comments = await getComments(user.id);
  return <div>...</div>;
}
```

Issue:

Sequential awaits block each other; posts and comments could load in parallel

Explanation: Each await blocks next line. posts and comments both need user.id but don't depend on each other.

Fix:

Use Promise.all like this:

`const [posts, comments] = await Promise.all([getPosts(user.id),getComments(user.id)])`

Question 26 INTERMEDIATE

What's the difference between export const dynamic = 'auto' vs 'force-dynamic'?

Answer:

'auto' lets Next.js decide; 'force-dynamic' forces dynamic rendering always

Options:

- **auto:** Default, Next.js optimizes automatically
- **force-dynamic:** Always server-render, no cache
- **force-static:** Always static, error on dynamic
- **error:** Error if any dynamic functions used

Question 27 ADVANCED

Why does this middleware cause infinite redirects?

```
export function middleware(request) {
  if (!request.cookies.get('auth')) {
    return NextResponse.redirect(new URL('/login', request.url));
  }
}

export const config = {
  matcher: '/:path*'
};
```

Issue:

Middleware runs on /login too, creating redirect loop

Explanation: Matcher includes all paths including /login. Redirecting to /login triggers middleware again.

Fix: Exclude /login from matcher or add condition: if (pathname !== '/login')

Question 28 ADVANCED

What's wrong with this useFormStatus implementation?

```
'use client'

import { useFormStatus } from 'react-dom';

export default function Form() {
  const { pending } = useFormStatus();

  return (
    <form action={submitAction}>
      <button disabled={pending}>Submit</button>
    </form>
  );
}
```

Issue:

useFormStatus must be called in child component of form, not same component

Explanation: useFormStatus hook reads context from parent form. Calling in same component as form doesn't work.

Fix: Extract button to separate component that calls useFormStatus

Question 29 ADVANCED

Why doesn't this POST request work correctly?

```
export async function POST(request) {
  const body = request.body;
  const data = await db.create(body);
  return NextResponse.json(data);
}
```

Error:

request.body is a ReadableStream, not parsed JSON

Explanation: Request body must be parsed using await request.json() or request.formData(), not accessed directly.

Fix: const body = await request.json() before using it

Question 30 ADVANCED

What's the issue with this error boundary?

```
'use client'

export default function Error({ error }) {
  return (
    <div>
      <h2>Error: {error.message}</h2>
      <pre>{error.stack}</pre>
    </div>
  );
}
```

Security Issue:

Exposing stack traces in production reveals sensitive information

Explanation: Stack traces contain file paths, dependencies, internal logic. Should never show in production.

Fix: Only show stack in dev: {process.env.NODE_ENV === 'development' && error.stack}

Question 31 INTERMEDIATE

What's the purpose of the maxDuration export in Route Handlers?

> **Answer:**
> Sets maximum execution time for serverless functions (in seconds)

Usage:

- export const maxDuration = 60
- Default varies by hosting platform
- Vercel Hobby: 10s, Pro: 60s, Enterprise: 900s
- Prevents timeout errors for long operations
- Also works in page.js files

Question 32 INTERMEDIATE

How do you implement middleware that only runs on specific paths?

> **Answer:**
> Use config.matcher to specify path patterns as string or array of strings

Matcher Patterns:

- **'/about':** Exact path match
- **'/dashboard/:path*':** All dashboard routes
- **['/api/:path*', '/auth/:path*']:** Multiple patterns
- Use negative lookahead: '/((?!api|_next/static).*)'

Question 33 ADVANCED

What's wrong with this dynamic metadata generation?

```
export async function generateMetadata({ params }) {
  const post = await getPost(params.id);

  if (!post) {
    notFound();
  }

  return {
    title: post.title
  };
}
```

Issue:

notFound() prevents metadata return; page shows default metadata

Explanation: notFound() throws error that breaks function execution. Metadata never returned.

Fix: Check for not found in page component, not generateMetadata

Question 34 INTERMEDIATE

How do you handle file uploads with Server Actions?

Answer:

Use formData.get() to access File objects from multipart form data

Implementation:

- Form needs encType="multipart/form-data"
- File objects have name, size, type properties
- Use arrayBuffer() or text() to read content
- Can upload to cloud storage (S3, Cloudinary)
- Validate file type and size before processing

About Me

Hi, I'm Yogesh,

I'm a Freelancer, Mentor, Full Stack Developer and an Industry/Corporate Trainer.

Specializing in: HTML, JavaScript, CSS, React, Next.js, Node.js, and related web technologies.

I love sharing my knowledge through articles and tutorials.

I usually write at: freeCodeCamp, dev.to, Medium, Hashnode

Check out my all the courses/ebooks/webinars.

Don't forget to follow me on LinkedIn to learn new things everyday.